

Zarr-Cesium: Client-side interactive 2D and 3D visualization of large multidimensional Zarr datasets in the browser

Tobias Ferreira ¹

¹ National Oceanography Centre, UK

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Environmental sciences increasingly rely on multidimensional datasets describing oceanic, atmospheric, and terrestrial systems across space, depth, and time. Numerical models and remote sensing products routinely produce terabyte-scale data cubes with dimensions such as longitude, latitude, elevation, and time (Eyring et al., 2016; Hersbach et al., 2020). These datasets are commonly stored in chunked, cloud-optimized formats such as Zarr (Zarr Developers, 2026b), enabling scalable storage and parallel analysis.

Although storage and computational frameworks have evolved to support these multidimensional data structures (Abernathey et al., 2021; Zarr Developers, 2026b), visualization workflows often transform them into two-dimensional raster products for web delivery, separating rendered outputs from the underlying data cube.

Zarr-Cesium is a TypeScript library that addresses these limitations. It enables interactive, browser-native visualization of multidimensional Zarr datasets within a 3D geospatial environment using CesiumJS (Cesium GS, 2026). The software streams chunked array data directly from object storage into the browser and renders it using WebGL2, without requiring backend services, tile-generation pipelines, or intermediate raster products.

The toolkit supports 2D scalar fields, 3D volumetric slicing, and animated vector fields. It is compatible with Zarr v2 and v3, multiscale pyramids (Carbonplan, 2026; Zarr Developers, 2026a), and CF-compliant metadata conventions (Eaton et al., 2025). By integrating cloud-native data access with GPU-accelerated rendering, Zarr-Cesium enables interactive exploration of large scientific datasets directly from their cloud-hosted storage.

Statement of Need

Although storage and computation frameworks have matured, visualization workflows remain fragmented across several partially overlapping ecosystems.

Scientific communication frequently relies on static figures or screenshots, which flatten inherently multidimensional systems into fixed two-dimensional views. Server-side tile-generation platforms convert datasets into raster products through Web Map Service (WMS) (Open Geospatial Consortium, 2006) or custom APIs, requiring persistent backend infrastructure, preprocessing steps, and operational maintenance. These workflows also duplicate data into derived tile products, increasing storage requirements and separating rendered output from its canonical source.

Three-dimensional visualization is frequently essential for scientific interpretation. Many environmental processes exhibit strong vertical structure, including ocean stratification and thermocline variability, mixed-layer dynamics, and large-scale overturning circulation (Pickard &

40 [Emery, 1990](#)). Atmospheric systems similarly depend on vertical structure, including boundary
41 layer evolution and convection ([Stull, 1988](#)). Horizontal maps alone can obscure depth-
42 dependent gradients, vertical shear, and subsurface anomalies that are central to physical
43 interpretation. Interpretation of velocity fields further requires contextualizing transport
44 pathways and circulation patterns across depth and time ([Marshall & Plumb, 1961](#)).

45 As multidimensional datasets become standard in operational modeling and reanalysis systems,
46 there is a need for visualization approaches that preserve their native structure rather than
47 flattening or duplicating them into derived products.

48 Statement of the Field

49 Existing visualization ecosystems address parts of this problem but do not provide a unified
50 client-side solution for multidimensional Zarr datasets.

51 Server-side tile-generation platforms convert multidimensional datasets into raster products
52 exposed through WMS endpoints or custom APIs. Systems such as GeoServer ([GeoServer,](#)
53 [2026](#)) and ncWMS ([Blower et al., 2013](#)) enable serving NetCDF-based datasets as map layers
54 but require persistent backend infrastructure and preprocessing steps to generate tiles.

55 Cloud-native raster ecosystems based on Cloud-Optimized GeoTIFFs (COGs) ([Open Geospatial](#)
56 [Consortium, 2023](#)) enable efficient streaming of large two-dimensional imagery products. Tools
57 such as TiTiler ([Development Seed, 2026a](#)) provide scalable server-side generation of map
58 tiles directly from COGs. However, these approaches are fundamentally oriented toward two-
59 dimensional raster data. Extending them to higher-dimensional data cubes typically requires
60 precomputed slicing, separate endpoints for each non-spatial dimension, or additional backend
61 logic to manage depth and time selection.

62 For Zarr datasets, backend tile-generation solutions such as titiler-xarray ([Development Seed,](#)
63 [2026b](#)) and xpublish-tiles ([Earthmover, 2026](#)) expose Zarr-backed arrays through tile APIs.
64 While these tools allow dynamic slicing of multidimensional data, they still depend on server-side
65 infrastructure to perform array access, rasterization, and tile generation. Visualization therefore
66 remains mediated by backend services, and rendered products are transmitted as image tiles
67 rather than structured array subsets.

68 Client-side geospatial engines such as CesiumJS ([Cesium GS, 2026](#)) provide powerful 3D globe
69 visualization but do not natively support streaming and slicing of multidimensional chunked
70 array formats such as Zarr. Conversely, browser-based Zarr libraries such as Zarrita ([Zarrita](#)
71 [Developers, 2026](#)) enable direct access to chunked arrays in JavaScript but do not provide
72 geospatial integration, volumetric slicing, or GPU-based colormapping pipelines.

73 Consequently, no general-purpose framework currently supports direct streaming of
74 multidimensional Zarr arrays from object storage, automatic interpretation of CF-compliant
75 metadata, dynamic slice computation in the browser, GPU-based rendering of scalar and
76 vector fields, and fully static deployment without backend services. Zarr-Cesium fills this gap
77 by integrating these capabilities into a coherent client-side architecture.

78 Software Design

79 Zarr-Cesium is implemented in TypeScript and organized around three primary provider classes:
80 ZarrLayerProvider for 2D scalar fields, ZarrCubeProvider for 3D volumetric slicing, and
81 ZarrCubeVelocityProvider for animated vector fields. These providers encapsulate data
82 access, multidimensional slicing logic, and rendering integration.

83 All providers access Zarr datasets via a browser-based FetchStore, supporting HTTP range
84 requests, consolidated metadata, and both Zarr v2 and v3 specifications. Multiscale pyramid
85 datasets are detected through multiscales metadata, and resolution levels are selected

86 dynamically based on view scale. A lightweight in-memory cache maintains recently accessed
87 multiscale levels.

88 Dimension identification is performed automatically using CF standard_name attributes (Eaton
89 et al., 2025) and configurable alias mappings. Geographic bounds from Cesium tiles or
90 camera extents are translated into array index ranges, with automatic CRS detection between
91 EPSG:4326 and EPSG:3857. Zarr slice arguments are constructed dynamically, and value-based
92 selectors (e.g., timestamps or depths) are converted to nearest-index selections.

93 To maintain interactivity, the system limits concurrent chunk requests and uses
94 AbortController instances to cancel stale requests during rapid navigation. Only
95 visible spatial subsets are requested, and multiscale level selection reduces resolution at broader
96 zoom levels.

97 Rendering is performed using WebGL2. For scalar fields, float32 array subsets are uploaded
98 as R32F textures and processed in fragment shaders that apply scale-factor correction,
99 normalization, no-data masking, and dynamic colormapping. Because scaling and colormapping
100 occur on the GPU, visual parameters can be adjusted without re-fetching data.

101 For volumetric cubes, array subsets are reshaped into ndarray structures and horizontal or vertical
102 slices are generated dynamically. Textured Cesium Primitive geometries are constructed, and
103 elevation values are converted into globe-compatible heights with optional vertical exaggeration
104 or subsurface offsets. Slices are regenerated when dimension selectors or styling parameters
105 change.

106 Velocity fields are handled by loading U and V components from separate Zarr stores, extracting
107 elevation slices, and passing the data to the cesium-wind-layer library (Cesium Wind Layer
108 Developers, 2024) for particle-based animation. Multiple elevation layers can be rendered
109 simultaneously, enabling stacked three-dimensional flow visualizations.

110 By combining selective chunk loading, multiscale resolution management, GPU-based rendering,
111 and minimal data duplication, the architecture enables interactive exploration of datasets
112 significantly larger than browser memory limits. Performance is constrained primarily by chunk
113 size and network bandwidth rather than total dataset size.

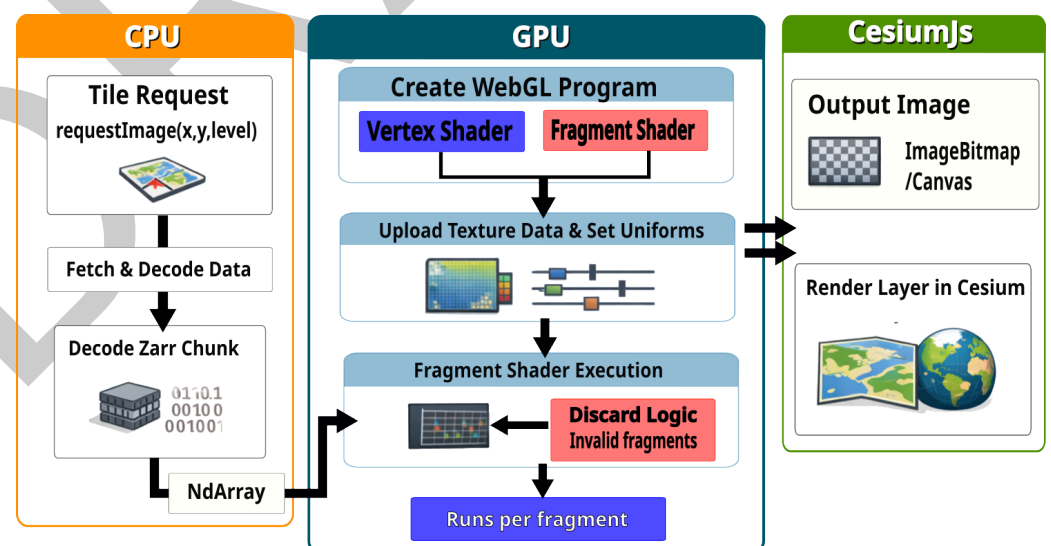


Figure 1: Representation of the workflow of the Zarr-Cesium package, from tile request to WebGL layer generation on the map.

Research Impact Statement

Zarr-Cesium integrates cloud-native chunked storage access, CF-aware dimension resolution, multiscale pyramid handling, and GPU-based rendering into a unified client-side architecture. The software was developed to meet an operational need for interactive visualization of multidimensional Zarr datasets stored in object storage without server-side infrastructure. It serves as the core rendering engine of the AtlantiS visualisation tool at the National Oceanography Centre, enabling browser-based exploration of multidimensional oceanographic datasets in a 3D geospatial environment through fully client-side streaming and rendering. It has been released as an open-source TypeScript library to enable broader adoption across the environmental sciences and related domains.

Zarr-Cesium fills a clear gap in the ecosystem. No existing framework integrates client-side multidimensional slicing, CF-aware metadata interpretation, WebGL2-based GPU rendering, and CesiumJS 3D geospatial visualization into a unified, backend-free system. Beyond reducing infrastructure complexity, this architecture supports more transparent and exploratory analysis of inherently multidimensional environmental systems. Interactive slicing and dynamic rendering allow datasets to be examined in their full spatial and temporal structure rather than as predefined two-dimensional views. As cloud-native storage formats such as Zarr v3 and GeoZarr gain adoption, coupling scalable object storage with browser-native visualization becomes an essential component of modern scientific data dissemination. Zarr-Cesium provides such a component, lowering barriers to interactive publication and enabling reproducible, infrastructure-light access to large environmental datasets.

AI usage disclosure

Portions of this manuscript were edited with the assistance of a large language model for clarity and structure. All technical content, software design decisions, and scientific claims were written and verified by the author. No AI-generated code was included in the software without manual review and testing.

Acknowledgements

This work builds upon the open-source efforts of the CesiumJS, Zarrita, and cesium-wind-layer communities. Storage and testing infrastructure were supported by JASMIN, the UK's environmental data analysis facility. The project forms part of the AtlantiS initiative led by the National Oceanography Centre.

References

- Abernathy, R., Hamman, J., Lepore, C., & others. (2021). Cloud-native repositories for big scientific data. *Computing in Science & Engineering*, 23(2), 26–35. <https://doi.org/10.1109/MCSE.2021.3059437>
- Blower, J. D., Gemmell, A. L., Griffiths, G. H., Haines, K., Santokhee, A., & Yang, X. (2013). A web map service implementation for the visualization of multidimensional gridded environmental data. *Environmental Modelling & Software*, 47, 218–224. <https://doi.org/https://doi.org/10.1016/j.envsoft.2013.04.002>
- Carbonplan. (2026). *Ndpyramid: Multiscale pyramids for multidimensional arrays*. <https://github.com/carbonplan/ndpyramid>
- Cesium GS. (2026). *CesiumJS*. <https://cesium.com/platform/cesiumjs/>

- 156 Cesium Wind Layer Developers. (2024). *Cesium-wind-layer*. [https://github.com/hongfaqiu/](https://github.com/hongfaqiu/cesium-wind-layer)
157 [cesium-wind-layer](https://github.com/hongfaqiu/cesium-wind-layer)
- 158 Development Seed. (2026a). *TiTiler: A modern dynamic tile server for raster data*. [https://](https://github.com/developmentseed/titiler)
159 github.com/developmentseed/titiler
- 160 Development Seed. (2026b). *Titiler-xarray*. [https://github.com/developmentseed/titiler-](https://github.com/developmentseed/titiler-xarray)
161 [xarray](https://github.com/developmentseed/titiler-xarray)
- 162 Earthmover. (2026). *Xpublish-tiles*. <https://github.com/earth-mover/xpublish-tiles>
- 163 Eaton, B., Gregory, J., Drach, B., Taylor, K., Hankin, S., Caron, J., Signell, R., Bentley,
164 P., Rappa, G., Höck, H., Pamment, A., Juckes, M., Raspaud, M., Blower, J., Horne,
165 R., Whiteaker, T., Blodgett, D., Zender, C., Lee, D., ... Laake, P. V. (2025). *NetCDF*
166 *climate and forecast (CF) metadata conventions* (Version 1.13). CF Community. [https://](https://doi.org/10.5281/zenodo.17801666)
167 doi.org/10.5281/zenodo.17801666
- 168 Eyring, V., Bony, S., Meehl, G. A., Senior, C. A., Stevens, B., Stouffer, R. J., & Taylor,
169 K. E. (2016). Overview of the coupled model intercomparison project phase 6 (CMIP6).
170 *Geoscientific Model Development*, 9, 1937–1958. [https://doi.org/10.5194/gmd-9-1937-](https://doi.org/10.5194/gmd-9-1937-2016)
171 [2016](https://doi.org/10.5194/gmd-9-1937-2016)
- 172 GeoServer. (2026). *GeoServer*. <https://geoserver.org>
- 173 Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horanyi, A., Muñoz-Sabater, J., Nicolas,
174 J., Peubey, C., Radu, R., Schepers, D., & others. (2020). The ERA5 global reanalysis.
175 *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. [https://](https://doi.org/10.1002/qj.3803)
176 doi.org/10.1002/qj.3803
- 177 Marshall, J., & Plumb, R. A. (1961). *Atmosphere, ocean and climate dynamics: An introductory*
178 *text* (Vol. 2). Academic Press.
- 179 Open Geospatial Consortium. (2006). *Web map service (WMS) implementation specification*.
180 <https://www.ogc.org/standards/wms>
- 181 Open Geospatial Consortium. (2023). *OGC Cloud Optimized GeoTIFF Standard* (J. Masó, Ed.;
182 OGC Implementation Standard No. 21-026; Version 1.0). Open Geospatial Consortium.
183 <https://docs.ogc.org/is/21-026/21-026.html>
- 184 Pickard, G. L., & Emery, W. (1990). *Descriptive physical oceanography: An introduction* (p.
185 320). ISBN: 075062759X
- 186 Stull, R. B. (1988). *An introduction to boundary layer meteorology*. Kluwer Academic
187 Publishers. <https://doi.org/10.1007/978-94-009-3027-8>
- 188 Zarr Developers. (2026a). *GeoZarr specification*. [https://github.com/zarr-developers/geozarr-](https://github.com/zarr-developers/geozarr-spec)
189 [spec](https://github.com/zarr-developers/geozarr-spec)
- 190 Zarr Developers. (2026b). *Zarr: Open standard for chunked, compressed n-dimensional arrays*.
191 <https://zarr.dev/>
- 192 Zarr Developers. (2026). *Zarrita: A minimal & modular zarr implementation in TypeScript*.
193 <https://github.com/manzt/zarrita.js>